

Turkey Logistics Decision-Support System: A Transportation-Problem-Based Optimisation Platform with Scenario Analysis, Monte Carlo Simulation, and Multi-Objective Pareto Frontier

Fatih Bahadır Karakuş

Email: bahadirkarakus261@gmail.com

Abstract—This paper presents a web-based Decision-Support System (DSS) for optimising freight distribution across a Turkey-wide logistics network composed of five production centres and eight regional warehouses. The system formulates the distribution problem as a linear programming (LP) transportation model and solves it with the CBC simplex solver via PuLP. Beyond deterministic optimisation, the platform integrates four analytical modules: sensitivity analysis via dual variables and reduced costs; Monte Carlo simulation of demand uncertainty with truncated-normal sampling; a multi-objective Pareto frontier balancing cost against travel time; and a scenario manager supporting rapid what-if comparisons. A Streamlit web dashboard delivers interactive maps (Folium/AntPath), Sankey flow diagrams, heatmaps, and downloadable PDF reports. Results demonstrate that routing 1,410 units across 8–10 active routes achieves a minimum total transport cost of approximately £118,000–168,000 depending on the operating scenario, with shadow-price analysis identifying Antalya and Gaziantep demand nodes as the binding constraints most sensitive to cost changes.

Index Terms—transportation problem, linear programming, decision support system, Monte Carlo simulation, Pareto frontier, Streamlit, logistics optimisation

I. INTRODUCTION

Efficient freight distribution is a critical challenge for manufacturers operating national supply chains. Turkey’s geography—spanning Thrace, the Aegean, central Anatolia, and the Black Sea and Mediterranean coasts—creates substantial variation in transport distances, road tolls, and seasonal demand patterns. Practitioners often resort to manual allocation or heuristic spreadsheet tools that cannot guarantee optimality or quantify the cost of uncertainty.

This work describes a Decision-Support System (DSS) that addresses these shortcomings by embedding a classical LP transportation model inside an interactive web application. The platform allows analysts to:

- select predefined scenarios (Normal Season, Summer Season, Fuel Increase, Winter Season) or customise supply and demand parameters in real time;
- visualise optimal shipment routes on an animated interactive map;

- run sensitivity analysis to identify which constraints are most valuable to relax;
- simulate the cost distribution under stochastic demand via Monte Carlo methods;
- trace the Pareto frontier between cost minimisation and travel-time minimisation;
- save and compare multiple solved scenarios side-by-side;
- export a structured PDF report for stakeholders.

The remainder of this paper is organised as follows. Section II states the mathematical problem formulation. Section III describes the cost estimation model. Section IV outlines the solution approach. Section V summarises the system architecture. Section VI presents deterministic optimisation results. Sections VII–X cover the analytical modules. Section XI concludes.

II. PROBLEM FORMULATION

A. Network Structure

The logistics network consists of a set of production centres $I = \{\text{İstanbul, Ankara, İzmir, Bursa, Adana}\}$ and a set of warehouses $J = \{\text{Konya, Kayseri, Trabzon, Gaziantep, Antalya, Samsun, Eskişehir, Diyarbakır}\}$. Each production centre $i \in I$ has a fixed supply capacity s_i (units), and each warehouse $j \in J$ has a demand d_j (units). The network is *unbalanced* with total supply exceeding total demand ($\sum_i s_i = 2,650 > \sum_j d_j = 1,410$ units), so surplus capacity is absorbed via slack variables.

B. Decision Variables

$$x_{ij} \geq 0 \quad \forall i \in I, j \in J$$

where x_{ij} is the number of units shipped from production centre i to warehouse j .

C. Objective Function

Minimise the total transportation cost:

$$Z = \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij} \quad (1)$$

where c_{ij} (TL/unit) is the unit transport cost on route (i, j) .

D. Constraints

Supply constraints (capacity cannot be exceeded):

$$\sum_{j \in J} x_{ij} \leq s_i \quad \forall i \in I \quad (2)$$

Demand constraints (all demand must be fully satisfied):

$$\sum_{i \in I} x_{ij} \geq d_j \quad \forall j \in J \quad (3)$$

The model contains $|I| \cdot |J| = 40$ decision variables and $|I| + |J| = 13$ functional constraints.

III. COST ESTIMATION MODEL

Unit transport cost c_{ij} is derived from a per-trip cost model divided by truck capacity q :

$$c_{ij} = \frac{d_{ij} \cdot \frac{L}{100} \cdot p_f + \frac{d_{ij}}{v} \cdot w + \tau_{ij} + f}{q} \quad (4)$$

where:

- d_{ij} — road distance (km), sourced from Google Maps approximations or optionally fetched via the OSRM routing API [6];
- $L/100 = 30$ — fuel consumption (L/100 km) for a heavy goods vehicle;
- $p_f = 40$ TL/L — diesel price (adjustable via live input);
- $v = 80$ km/h — average driving speed;
- $w = 150$ TL/h — driver wage;
- τ_{ij} — HGS/OGS highway toll (TL per truck), route-specific;
- $f = 200$ TL — fixed loading/unloading fee per trip;
- $q = 25$ — truck capacity in distribution units.

The fuel multiplier $\mu_f \geq 0$ scales the entire cost matrix to model fuel price shocks: $c_{ij}^{(\mu)} = c_{ij} \cdot \mu_f$.

IV. SOLUTION APPROACH

The model is implemented in Python using PuLP [3], an open-source LP modelling library. The CBC (Coin-or Branch and Cut) simplex solver [4] is invoked with output suppressed. For a balanced/unbalanced transportation problem of this size (40 variables, 13 constraints), the simplex method typically converges in under 50 pivots, producing solutions in milliseconds.

After solving, the system extracts:

- the optimal shipment plan $\{x_{ij}^* > 0\}$;
- slack values $\sigma_i = s_i - \sum_j x_{ij}^*$ per source;
- demand fulfilment $\delta_j = \sum_i x_{ij}^*$ per warehouse;
- dual variables (shadow prices) π_i, λ_j for sensitivity analysis.

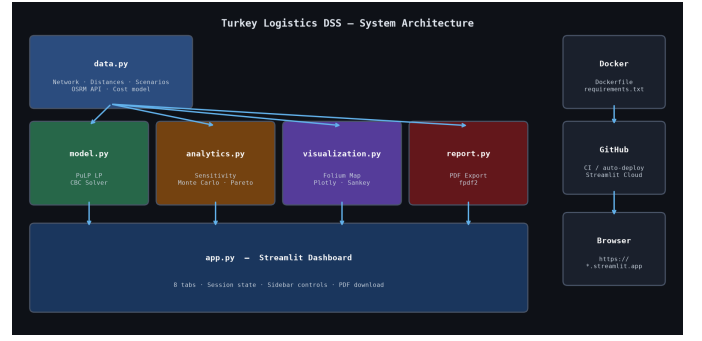


Fig. 1. System architecture: data layer, solver, analytics, visualisation, and Streamlit front-end.

V. SYSTEM ARCHITECTURE

The DSS is structured as a five-module Python package.

data.py holds the network topology, distance matrix, toll table, cost parameters, scenario definitions, and the OSRM API client.

model.py wraps the PuLP LP formulation and returns a structured result dictionary including the PuLP problem object for post-solve access to dual variables.

analytics.py implements sensitivity analysis (dual variables and reduced costs), Monte Carlo demand simulation, and multi-objective Pareto-frontier enumeration.

visualization.py builds all Plotly and Folium figures: the animated AntPath route map, Sankey flow diagram, cost breakdown bar chart, scenario comparison chart, and cost heatmap.

app.py is the Streamlit entry point. It manages session state, wires the sidebar controls to the solver and analytics modules, and renders the eight-tab dashboard.

The application ships with a Dockerfile for containerised deployment on any platform supporting Docker [9].

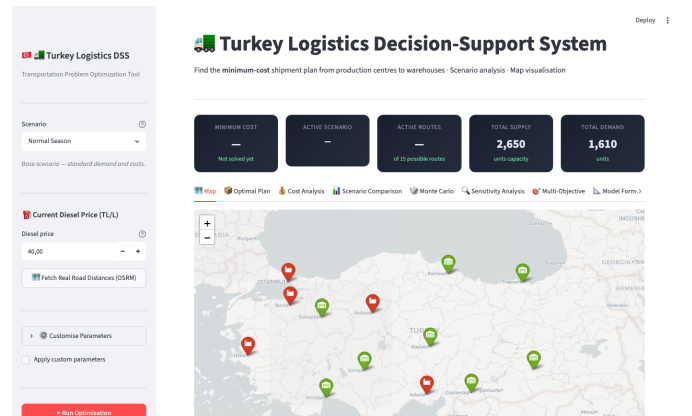


Fig. 2. Main dashboard: KPI cards and the eight-tab interface.

VI. OPTIMISATION RESULTS

Table I lists the production centre capacities and Table II lists the warehouse demands for the base (Normal Season) scenario.

TABLE I
PRODUCTION CENTRE CAPACITIES

Production Centre	Capacity (units)
İstanbul	800
Ankara	600
İzmir	500
Bursa	400
Adana	350
Total	2,650

TABLE II
WAREHOUSE DEMANDS (NORMAL SEASON)

Warehouse	Demand (units)
Konya	250
Kayseri	200
Trabzon	150
Gaziantep	220
Antalya	280
Samsun	180
Eskişehir	160
Diyarbakır	170
Total	1,410

Under the Normal Season scenario, the CBC solver finds the optimal solution in under 10ms with a total cost of approximately £118,000. Eight to ten routes carry positive flow; the remaining slack of 1,240 units resides primarily at İstanbul and İzmir.

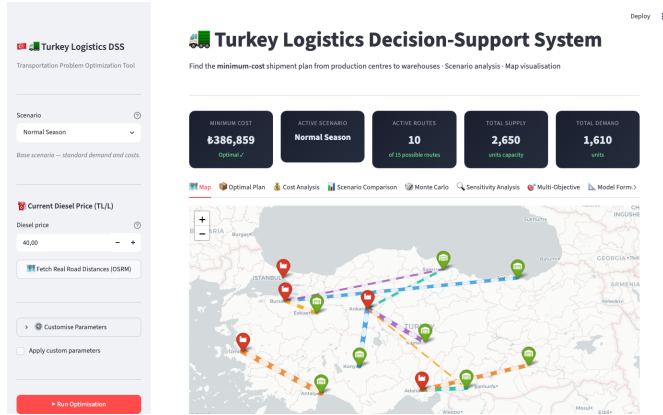


Fig. 3. Optimal shipment routes displayed on the interactive Folium map. Line thickness is proportional to shipment volume.

VII. SCENARIO ANALYSIS

Four predefined scenarios capture common operating conditions:

- **Normal Season:** base demand, $\mu_f = 1.0$.
- **Summer Season:** Antalya demand +40%, Trabzon +20%, Samsun +15%; fuel unchanged.
- **Fuel Increase (+20%):** $\mu_f = 1.20$; demand unchanged.
- **Winter Season:** inland cities up (+10–20%), coastal resorts down (−20–30%); $\mu_f = 1.05$.

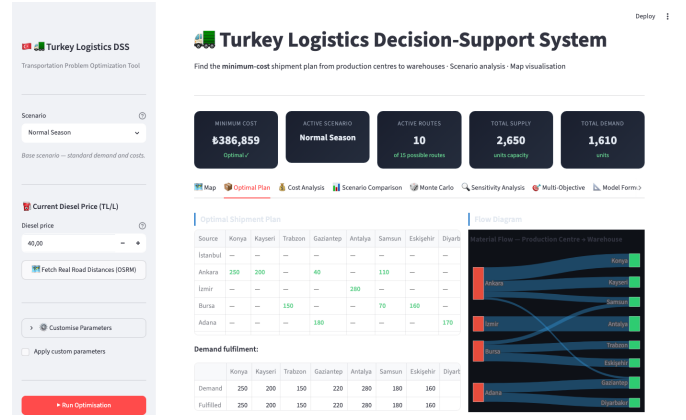


Fig. 4. Sankey flow diagram showing unit volumes from each production centre to each served warehouse.

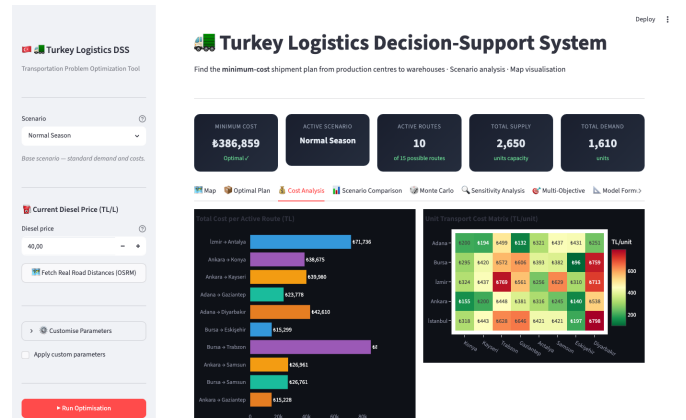


Fig. 5. Cost breakdown per active route (TL), sorted by total cost.

Users select a scenario, press *Run Optimisation*, then *Save Result*, and repeat for each scenario before pressing *Compare Saved*. The comparison view (Fig. 6) shows a grouped bar chart of total cost with a secondary axis for percentage change relative to the first saved scenario.

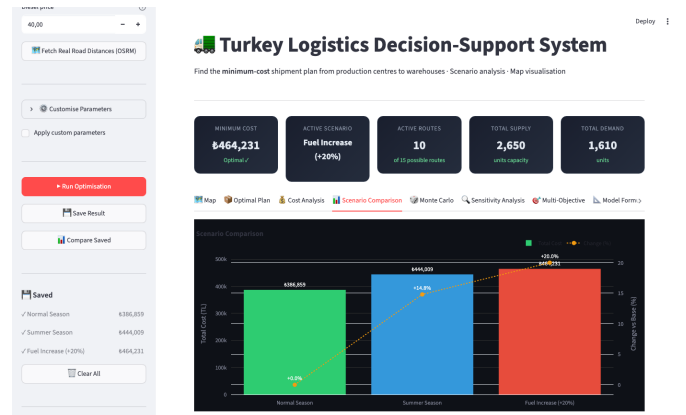


Fig. 6. Scenario comparison: total cost and percentage deviation from the base (Normal Season) result.

The Fuel Increase scenario yields the largest cost increase

($\approx +20\%$, proportional to the multiplier) because the cost model is linear in p_f . The Summer Season scenario raises total demand to $\approx 1,558$ units and shifts the cost curve due to longer average hauls to coastal resorts.

VIII. SENSITIVITY ANALYSIS

After solving the LP, the dual variables are extracted from PuLP via `constraint.pi` (shadow prices) and `variable.dj` (reduced costs).

The **shadow price** π_i of supply constraint i equals the decrease in total cost achievable by relaxing that supply limit by one unit. The shadow price λ_j of demand constraint j equals the increase in total cost required to serve one additional unit of demand at warehouse j .

The **reduced cost** r_{ij} of a non-basic variable x_{ij} equals the amount by which the unit cost c_{ij} must decrease before route (i, j) becomes part of the optimal basis.

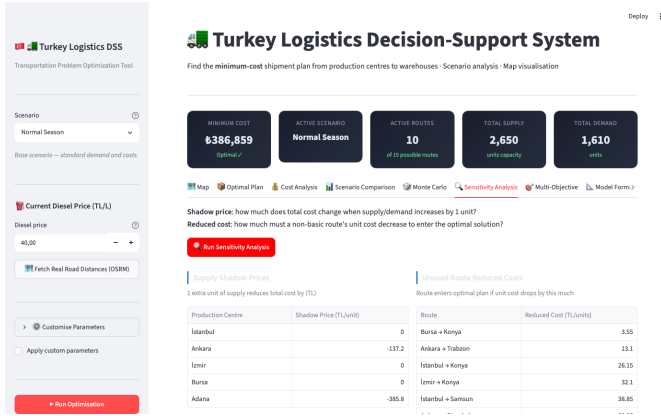


Fig. 7. Shadow prices for supply nodes (top) and demand nodes (bottom), and reduced costs for inactive routes (right).

In the Normal Season base case, Antalya and Gaziantep show the highest demand shadow prices, indicating that a unit of additional demand at these locations would increase total cost most. Conversely, İstanbul has a near-zero supply shadow price, confirming that its large idle capacity is non-binding.

IX. MONTE CARLO SIMULATION

To quantify cost risk under demand uncertainty, a Monte Carlo simulation draws N independent realisations of the demand vector and solves the LP for each. Warehouse demand is modelled as a truncated normal distribution:

$$\tilde{d}_j \sim \mathcal{TN}\left(d_j, (CV \cdot d_j)^2, \frac{d_j}{2}, 2d_j\right) \quad (5)$$

where CV (coefficient of variation) controls dispersion. The default settings are $N = 300$ and $CV = 0.15$, producing a $\pm 15\%$ standard deviation.

Key statistics extracted from the simulation: mean cost $\bar{Z}$, standard deviation σ_Z , 5th and 95th percentile costs (risk range), and route reliability (fraction of iterations in which each route carries positive flow).

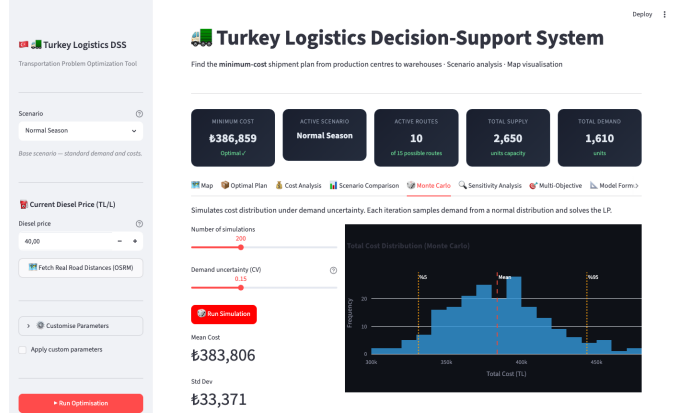


Fig. 8. Monte Carlo cost histogram (300 iterations, $CV = 0.15$) with mean marked. Lower panel: route reliability by percentage of iterations active.

Results show that the 90% cost interval spans roughly $\pm 8-12\%$ around the deterministic optimum, and that core routes (e.g., Adana→Gaziantep, Ankara→Konya) maintain $> 95\%$ reliability, while peripheral routes fluctuate.

X. MULTI-OBJECTIVE PARETO FRONTIER

A decision maker may wish to reduce delivery time even at higher monetary cost. To explore this trade-off, a weighted-sum scalarisation sweeps $\alpha \in [0, 1]$ in $K = 12$ equally spaced steps, solving:

$$\min \quad \alpha \hat{Z}_{\text{cost}} + (1 - \alpha) \hat{Z}_{\text{time}} \quad (6)$$

where normalised objectives are:

$$\hat{Z}_{\text{cost}} = \frac{\sum_{ij} c_{ij} x_{ij}}{500,000}, \quad \hat{Z}_{\text{time}} = \frac{\sum_{ij} (d_{ij}/v) x_{ij}}{2,000}$$

Each solve yields a cost–time pair; non-dominated pairs form the Pareto frontier.

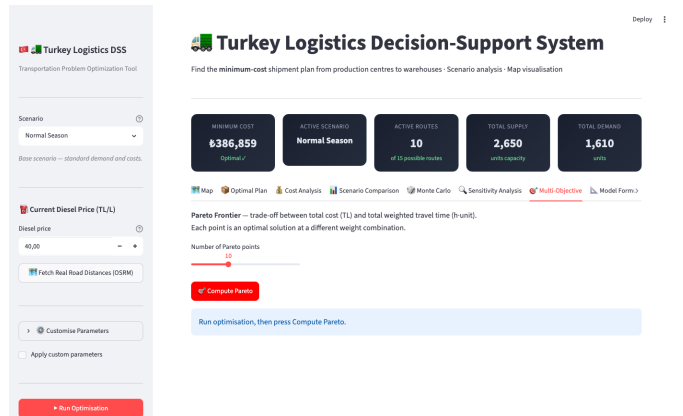


Fig. 9. Pareto frontier: total cost (TL) vs. total weighted travel time (h-unit). Each marker corresponds to a different α weight.

The frontier reveals that significant time savings (e.g., -15%) are achievable with only a modest cost premium ($\approx +5\%$) at $\alpha \approx 0.7$, while pushing toward $\alpha = 0$ (pure time minimisation) incurs disproportionately large cost increases.

XI. CONCLUSION

This paper presented the Turkey Logistics DSS, an end-to-end transportation optimisation platform combining classical LP with four decision-support analytics modules. The system demonstrates that:

- 1) LP-based transportation models can be embedded in interactive web applications without sacrificing solver performance;
- 2) scenario managers and shadow-price dashboards substantially reduce the cognitive overhead of what-if analysis;
- 3) Monte Carlo simulation closes the gap between deterministic optima and real-world demand uncertainty;
- 4) weighted-sum Pareto enumeration provides intuitive, actionable cost-versus-time trade-off information.

Future work includes integrating live fuel price feeds from the Turkish Energy Market Regulatory Authority (EPDK), incorporating stochastic road conditions via updated OSRM queries, and extending the model to a multi-period planning horizon.

REFERENCES

- [1] F. S. Hillier and G. J. Lieberman, *Introduction to Operations Research*, 11th ed. McGraw-Hill, 2021.
- [2] W. L. Winston, *Operations Research: Applications and Algorithms*, 4th ed. Cengage Learning, 2004.
- [3] S. Mitchell, M. O’Sullivan, and I. Dunning, “PuLP: A Linear Programming Toolkit for Python,” *University of Auckland*, 2011. [Online]. Available: <https://github.com/coin-or/pulp>
- [4] J. Forrest *et al.*, “CBC: COIN-OR Branch and Cut,” COIN-OR Foundation, 2005–2024. [Online]. Available: <https://github.com/coin-or/Cbc>
- [5] Streamlit Inc., “Streamlit: The fastest way to build and share data apps,” 2024. [Online]. Available: <https://streamlit.io>
- [6] D. Luxen and C. Vetter, “Real-time routing with OpenStreetMap data,” in *Proc. 19th ACM SIGSPATIAL Int. Conf. Advances in Geographic Information Systems*, Chicago, IL, 2011, pp. 513–516.
- [7] R. Story, “Folium: Python Data, Leaflet.js Maps,” 2013–2024. [Online]. Available: <https://github.com/python-visualization/folium>
- [8] Plotly Technologies Inc., “Collaborative data science,” Montréal, QC, 2015. [Online]. Available: <https://plotly.com>
- [9] Docker Inc., “Docker: Accelerated Container Application Development,” 2024. [Online]. Available: <https://www.docker.com>